Lesson 8: Visibility, Lighting, Reflections & Shading

Lesson 8: Visibility, Lighting, Reflections & Shading 1
8.1 Visibility in Graphics Theory 2
8.2 Visibility in OpenGL 2
8.2.1 Discussions 2
8.3 Simple Reflection Models 2
8.4 Lighting in OpenGL 3
8.5 Flat & Interpolative Shading 4
8.6 Shading in OpenGL 4
Lesson 8 Discussion Session 5

8.1 Visibility in Graphics Theory

Correct visibility – when multiple opaque polygons cover the same screen space, only the front most one is visible (remove the hidden surfaces) wrong visibility

8.2 Visibility in OpenGL

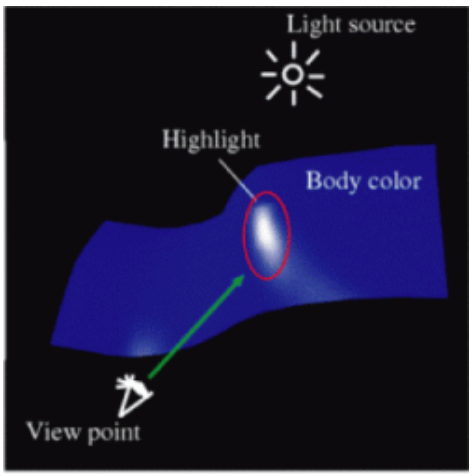
8.2.1 Discussions

OpenGL directly supports depth buffering, but it is often used in addition to other visibility techniques in interactive applications.
For example, many games use a BSP tree to prune the amount of static map geometry that is processed that would otherwise not be visible anyway. Also, when dealing with blended, translucent materials, these objects often must be drawn from back to front without writing to the depth buffer to get the correct appearance.
For simple scenes, however, the depth buffer alone is sufficient.
To use depth buffering in OpenGL with GLUT, the OpenGL context must be initialized with memory allocated for a depth buffer, with a command such as glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH);
Next, depth writing and testing must be enabled in OpenGL:
glEnable(GL_DEPTH_TEST);
OpenGL will automatically write pseudodepth values to the depth buffer when a primitive is rendered as long as the depth test is enabled.
The glDepthMask function can be used to disable depth writes, so depth testing will occur without writing to the depth buffer when rendering a primitive.
When clearing the display to render a new frame, the depth buffer should also be cleared:
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

8.3 Simple Reflection Models

Surface Reflection depends on both the viewing and illumination direction.A Simple Reflection Model -Dichromatic Reflection

Observed Image Color = a x Body Color + b x Specular Reflection ColorR



Does not specify any specific model forDiffuse/specular reflection

8.4 Lighting in OpenGL

OpenGL provides functions for

– setting up point light sources and spotlights,

– selecting surface reflection coefficients

– choosing values for several parameters in the basic illumination model

Implements the Phong reflection model

Steps in OpenGL Lighting

1. Enable lighting and select shading model

2. Specify normals

3. Specify material properties

4. Specify lights

Enable Lighting Calculations

Lighting calculations are enabled globally in an OpenGL program using

glEnable (GL_LIGHTING);

Once lighting is enabled, glColor*() ignored

Enable Individual Light Sources

Up to eight light sources (GL_MAX_LIGHTS) can be included in a standard OpenGL implementation

–Some implementations may allow more than 8

Each light source is referenced by its identifier:

GL_LIGHT0, GL_LIGHT1, ..., GL_LIGHT7

A light source is enabled with: glEnable (lightName);

Example: glEnable (GL_LIGHT0);

In OpenGL a viewer is replaced with projection plane. Rays which reach COP after

passing through viewing plane are actually seen. Color of pixels is determined by our interaction model

Limitations of OpenGL Lighting

- ✓ The pipeline is essentially one-way (left to right on the illustration).

- ✓ Once a polygon has been rendered, it's forgotten and can't be used as part of any other lighting calculations.
- ✓ No mirrors, glow sticks made with emissive light won't actually light up anything else in the scene, etc.
- ✓ Can only capture light source → o

8.5 Flat & Interpolative Shading

Flat Shading

Simplest, Cheapest, Fastest Shading Method

- Works well for objects really made of flat faces.
- Appearance depends on number of polygons for curved surface objects.
- Fills an entire polygon with one color intensity
- This model is only valid (realistic) if:
- The light source is imagined to be at infinity
- The viewer is at infinity
- The polygon is not an approximation to a curved surface

Flat Shading in OpenGL

Specify using
glShadeModel(GL_FLAT);
The color at the first vertex will determine the shade of the whole polygon.
Flat shading suffers from "mach band effect" human eyes accentuate the discontinuity at the boundary

Interpolative Shading

The idea of **interpolative shading** is to avoid computing the full lighting equation at each pixel by interpolating quantities at the vertices of the faces.
There are two methods used for interpolative shading:
Gouraud Shading The radiance values are computed at the vertices and then linearly interpolated
within each triangle.
Phong shading The normal values at each vertex are linearly interpolated within each triangle, and the radiance is computed at each pixel.
Gouraud shading is more efficient, but Phong shading is more accurate.

8.6 Shading in OpenGL

OpenGL only directly supports Gouraud shading or flat shading.
Gouraud is enabled by default,computing vertex colors, and interpolating colors across triangle faces.
Flat shading can be enabled with glShadeModel(GL FLAT). This renders an entire face with the color of a single vertex, giving a faceted appearance.
With *pixel shaders* on programmable graphics hardware, it is possible to achieve Phong shading

by using a small program to compute the illumination at each pixel with interpolated normals. It is even possible to use a *normal map* to assign arbitrary normals within faces, with a pixel shader using these normals to compute the illumination.

Lesson 8 Discussion Session

How will you create an environment map to capture lighting effects from a real-world outdoor scene? (Think of Paul Debevec's "Rendering with Natural Light" scene.) Assume only tools that you may have in your home (e.g., a camera).